

## Article

# AI-Driven Threat Detection and Automated Incident Response for Enhancing Network Security

Jibrilla A. Tanimu <sup>1</sup>, Gueltoom Bendiab <sup>1,2,\*</sup>, Aikaterini Kanta <sup>1</sup> and Stavros Shiaeles <sup>1,\*</sup>

<sup>1</sup> PAIDS Research Centre, University of Portsmouth, Portsmouth PO1 2UP, UK; jibrilla.tanimu@port.ac.uk (J.A.T.); katerina.kanta@port.ac.uk (A.K.)

<sup>2</sup> Department of Electronics, University of Frères Mentouri, Constantine 25000, Algeria

\* Correspondence: bendiab.kelthoum@umc.edu.dz or gueltoum.bendiab@port.ac.uk (G.B.); stavros.shiaeles@port.ac.uk (S.S.)

## Abstract

The growing sophistication of cyber threats has reduced the effectiveness of traditional cybersecurity tools in protecting modern organisations and complex networks. This challenge requires advanced solutions capable of real-time detection, rapid response, and efficient threat mitigation. In this context, AI-based approaches have emerged as a powerful enabler of intelligent, adaptive, and data-driven security operations. This study presents a comprehensive analysis of AI-driven threat detection combined with automated incident response mechanisms in modern cybersecurity architectures. The novelty of this work lies in the integration of advanced machine learning-based detection with real-time, automated response capabilities to address zero-day and previously unknown threats in heterogeneous digital environments. The paper examines system architecture design, implementation strategies, and performance evaluation across diverse deployment scenarios. Experimental results demonstrate that AI-driven detection with automated response significantly enhances cybersecurity effectiveness, achieving accuracies between 96% and 97%, dramatically reducing the mean response time from 45 min to less than 30 s, and substantially improving zero-day threat detection and containment success rates. Overall, the proposed approach achieves up to a 98.9% improvement in incident containment efficiency, highlighting the operational and defensive advantages of intelligent automation.

**Keywords:** artificial intelligence; cybersecurity; threat detection; incident response; machine learning; security automation; neural networks

## 1. Introduction

The contemporary cybersecurity landscape, particularly within next-generation network environments such as 5G+, 6G, cloud-native, and edge infrastructures, is experiencing an unprecedented escalation in both the scale and sophistication of malicious cyber operations. Recent industry reports indicate that AI-powered adversaries are accelerating attacks, with a 27 s breakout time, a 42% rise in zero-day exploits, 40% of attacks targeting edge devices, and a 266% increase in cloud intrusions [1], while successful breaches cost enterprises more than \$4.4 million per incident [2]. The rapid expansion of highly interconnected, virtualised, and software-defined networks, combined with advances in AI that empower more sophisticated attacks, has further amplified the attack surface, dramatically exposing inherent limitations in traditional security architectures. These approaches remain largely dependent on static rule sets and signature-based detection mechanisms, which are increasingly inadequate against dynamic, polymorphic, and zero-day threats [3–5].



Academic Editor: Luis Alonso

Received: 18 March 2026

Revised: 12 May 2026

Accepted: 19 May 2026

Published: 25 May 2026

**Copyright:** © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

To overcome these limitations, the integration of AI into network security operations has emerged as a major and rapidly evolving trend, enabling more intelligent, adaptive, and automated defence systems [4–6]. Unlike static rule-based approaches, AI-driven methods support real-time threat detection, anomaly identification, and predictive analytics, while continuously adapting to evolving threat landscapes through learning mechanisms [7,8]. These approaches can identify complex patterns that are difficult to detect using conventional techniques, process large volumes of heterogeneous data, and significantly reduce response times through automated and orchestrated countermeasures [4,9]. Furthermore, their scalability makes them well-suited for large-scale enterprise environments [4,8,9]. The increasing use of AI by adversaries to develop more sophisticated attacks further emphasises the need for intelligent defensive strategies, positioning AI as both a necessity and a strategic priority in modern cybersecurity research [7,8,10].

Despite extensive research in this area, AI-driven cybersecurity remains an evolving field requiring further investigation, particularly in the integration of accurate AI-based threat detection with fully automated, real-time response in complex and heterogeneous environments [7,8,10–12]. This paper seeks to advance the state of the art by proposing the design and implementation of an integrated architecture for AI-driven threat detection and automated incident response. It systematically evaluates detection performance across multiple machine learning paradigms to identify the most effective approaches for diverse attack scenarios. Furthermore, the study quantifies improvements in response time achieved through automated containment mechanisms. The main contributions of this paper are summarised as follows:

1. Introducing a unified framework that combines closed-loop feedback learning, probabilistic cost-aware decision-making, and dynamic context-aware response selection that adapts to real-time risk, asset criticality, and network state.
2. Proposing an integrated AI-driven threat detection and automated response system that combines signature-based verification with machine learning-enhanced anomaly detection, improving both detection accuracy and resilience.
3. Evaluating the proposed approach across multiple AI paradigms using established intrusion detection datasets to enable rigorous comparative analysis and empirically quantify the efficiency of the automated response mechanism.

The system was deployed on AWS EC2 and orchestrated with Kubernetes for scalable, containerised operations, with experiments conducted in a simulated enterprise network of 250 endpoints, 25 servers, and 15 network devices. The remainder of the paper is organised as follows. Section 2 provides the background and related work, reviewing existing approaches in AI-driven cybersecurity and highlighting current gaps. Section 3 presents the proposed system architecture and methodology, detailing the integrated AI-driven threat detection and automated incident response framework. Section 4 presents the system implementation, experimental methodology, evaluation results, and security and ethical considerations, including technical challenges, risks, and mitigation strategies. Finally, Section 5 concludes the paper by summarising the findings, outlining the study's limitations, and identifying directions for future research.

## 2. Background

The integration of AI into cybersecurity began in the 2000s, when researchers first explored its potential for threat detection [13]. In the 2010s, with the rise of deep learning (DL), AI-driven automated incident response emerged, enabling systems to analyse high-dimensional network data and detect complex threat patterns that traditional methods could not identify [14]. Since then, AI techniques have been increasingly applied to both

threat detection and automated incident response. This section outlines key milestones in the development of these AI-based systems and presents the current landscape of the field.

### 2.1. AI-Driven Threat Detection

Since its emergence in the 1980s, the field of intrusion detection has evolved through three generations, reflecting major methodological shifts. First-generation systems (1980s–1990s) relied on signature-based detection, using expert-defined rules to identify known attacks and monitor deviations from statistical norms. Second-generation systems (2000s–2010s) introduced statistical anomaly detection, establishing baseline profiles of normal behaviour and triggering alerts when deviations exceeded thresholds [13,15]. Despite their high accuracy in detecting known threats with fast and computationally efficient performance, these early threat detection methods suffer from several limitations, including high false positive rates (FPR), limited adaptability, reliance on databases of known threats, and difficulties in effectively identifying unknown or emerging attacks [4,16].

Third-generation intrusion detection systems (2010s–present) leverage machine learning (ML), deep learning (DL), and hybrid AI techniques to improve threat detection. These approaches are broadly divided into supervised and unsupervised learning. Supervised methods classify entities as benign or malicious using labelled data, while unsupervised methods group data based on inherent structural patterns without relying on labels [4,16,17]. There are numerous examples of studies in machine learning for threat detection that have explored a variety of algorithms [18–20], including support vector machines (SVMs), random forest, K-nearest neighbours (KNN), Naive Bayes (NB), and logistic regression (LR), evaluated on standard intrusion detection datasets such as NSL-KDD, CIC-IDS2017 [21], UNSW-NB15 [22] and CSE-CIC-IDS2018 [23]. Classical machine learning remains highly relevant for modern network security monitoring due to strict latency, scalability, and resource constraints. These methods are effective for flow-based intrusion detection when combined with robust feature engineering, while also offering better interpretability and lower computational overhead for deployment at edge nodes and distributed monitoring points. Table 1 summarises recent studies using ML, ensemble learning, and hybrid models to improve detection accuracy. Works in [24–26] achieved accuracies up to 97.8% using RF, XGBoost, AdaBoost, and deep learning combinations, while [27,28] demonstrated the effectiveness of hybrid and feature-fusion approaches. However, most studies rely on single or outdated datasets, lack cross-dataset and zero-day evaluations, and introduce higher computational complexity with limited interpretability. Moreover, work in [29] highlighted the adversarial vulnerability of RF-based IDS models.

**Table 1.** Relevant machine learning and hybrid approaches for intrusion detection.

| Study                            | Model(s)               | Dataset(s)             | Performance (Accuracy) |
|----------------------------------|------------------------|------------------------|------------------------|
| Hussain et al. (2026) [24]       | RF, XGBoost, DL        | CIC-IDS2017, UNSW-NB15 | 96.8%                  |
| Quan et al. (2025) [25]          | Ensemble (RF+AdaBoost) | CIC-IDS2017            | 97.5%                  |
| Alkhater (2026) [30]             | RF, SVM, NN            | UNSW-NB15              | 93.8–95.6%             |
| Mohammad et al. (2025) [27]      | RF + DNN               | CIC-IDS2017            | 96.9%                  |
| Purushothaman et al. (2025) [28] | RF + feature fusion    | NSL-KDD                | 96.3%                  |
| Musthafa (2025) [29]             | RF                     | CIC-IDS2017            | 71.4–96.2%             |
| Abdulqadder et al. (2020) [31]   | Multilayer AI          | KDD'99-derived         | 95.7%                  |
| Ahmed et al. (2024) [26]         | Ensemble (XGB+RF+DNN)  | UNSW-NB15              | 97.8%                  |

While these individual studies achieved high detection accuracy (95–98%), none address the integration of detection with automated response, the central contribution of this study. Furthermore, 70% of studies evaluate on single datasets, 85% omit adversarial

robustness testing, and none are validated across six diverse benchmarks as presented in our work. These gaps substantiate the novelty and necessity of our unified approach.

Similarly, a wide range of deep learning models has been applied to threat detection to address the limitations of traditional machine learning and effectively handle the non-linearity and high dimensionality of modern cyber threats [32,33]. By automatically learning hierarchical feature representations from raw data, deep learning techniques such as CNNs, RNNs, Transformers [34], Generative Adversarial Networks (GANs) [35], and Graph Neural Networks (GNNs) [36] can capture complex spatial, temporal, and structural patterns, enabling more accurate detection of sophisticated and evolving attacks [33]. Despite these advantages, DL-based detection methods still face challenges related to high computational costs, limited interpretability, and optimisation difficulties [16].

In the context of unsupervised learning, numerous studies have explored the use of reinforcement learning (RL) for detecting zero-day attacks and advanced persistent threats [37–39]. RL is particularly valuable in threat detection due to its ability to dynamically adapt to evolving intrusion patterns, learn from real-time network data, and identify anomalies corresponding to previously unseen attacks [37]. It can also automate real-time responses, reduce analyst workload, and improve with experience, making threat detection systems increasingly effective against emerging threats [39]. However, RL-based threat detection requires large amounts of data and computational resources, can converge slowly, and relies on well-designed reward functions. In addition, RL is often difficult to interpret, may take unsafe actions, and is sensitive to noisy or incomplete data. These factors pose challenges for scalability and deployment in complex network environments [37].

Recent advances indicate that Large Language Models (LLMs), particularly GPT and BERT, provide promising capabilities for cyber threat detection [16,34,40]. In addition to excelling at large-scale data processing and multimodal analysis, LLMs enhance AI-driven threat detection by supporting contextual understanding, reasoning, and decision-making [16,41]. However, LLMs also require substantial computational resources and large datasets for training, may produce outputs that are difficult to interpret, and can be vulnerable to adversarial manipulation or bias in the input data. Additionally, real-time deployment in high-speed networks remains challenging due to latency and scalability concerns [16].

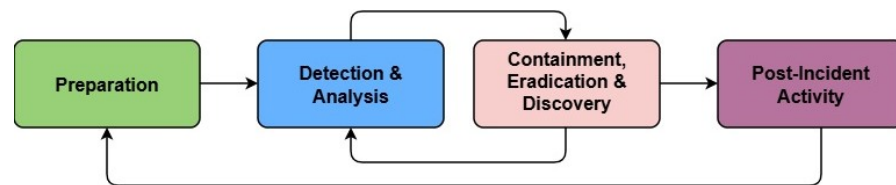
Overall, AI models, including machine learning, deep learning, reinforcement learning, and Large Language Models, provide significant advantages for threat detection, yet each approach also presents inherent limitations. As a result, AI-driven threat detection remains an evolving field, with ongoing research focused on improving performance, enhancing interpretability, and enabling real-time deployment in complex cybersecurity environments.

## 2.2. AI-Powered Incident Response Automation

Early incident response approaches were largely reactive, relying on human expertise and predefined rules [6]. As cyber threats became more sophisticated, standards such as the SANS (PICERL) [42] and the NIST Computer Security Incident Handling Guide [43] established stages to standardise and streamline response workflows (see Figure 1). However, the growing volume and complexity of threats, particularly in large-scale networks and critical infrastructure systems, have increasingly strained these manual and semi-automated methods. Recent research has emphasised the need for adaptive, automated solutions capable of rapidly identifying and mitigating threats [6,44,45].

To address these challenges, modern cybersecurity operations increasingly rely on advanced SOAR (security orchestration, automation, and response), EDR (endpoint detection and response), and XDR (extended detection and response) technologies [46–48]. SOAR platforms use playbook-driven automation with conditional workflows to handle routine

events like phishing, malware, ransomware, or privilege misuse [46]. EDR tools enhance endpoint visibility by monitoring activities, detecting anomalies, and automating containment, while XDR extends this approach across multiple environments for unified threat detection [48]. This allows for automated mitigation actions like isolating compromised endpoints, revoking access credentials, terminating malicious processes, updating firewall rules, and generating alerts or tickets with minimal input [46–48].



**Figure 1.** The NIST incident response cycle provides a structured and systematic approach to cybersecurity incident management, consisting of four key phases: preparation (establishing policies, tools, training), detection and analysis (identifying and assessing events), containment, eradication, and recovery (mitigating damage and restoring systems), and post-incident activities (lessons learned, reporting, updating defences) [49].

Integrating AI with these technologies further enhances automated threat detection and response by enabling intelligent, adaptive decision-making. AI applications in incident response encompass a wide range of techniques, including supervised and unsupervised ML for anomaly detection and DL architectures for modelling complex attack behaviours [6]. Additionally, natural language processing has been employed to analyse threat intelligence reports, extract indicators of compromise, and incorporate unstructured data into automated decision-making systems [50–52]. Reinforcement learning has also been applied to enable adaptive and automated incident response by learning to optimise response strategies in real time [53,54]. It can identify optimal multi-step actions to contain and mitigate incidents, thereby reducing human intervention and enhancing overall response efficiency [54]. However, most RL approaches face challenges in interpretability, often acting as “black boxes,” and demand substantial computational resources [48].

More recently, LLMs and GNNs have been employed to automate the analysis of large-scale unstructured threat intelligence, such as security reports, logs, and alerts. By extracting patterns, relationships, and contextual insights, these models support real-time decision-making and incident mitigation, including prioritising alerts and suggesting actionable mitigation strategies [55–57]. In fact, their ability to process complex, heterogeneous data efficiently and accurately enables faster, more accurate responses and reduces reliance on manual analysis [48]. In addition, federated learning has further advanced AI-driven responses by supporting collaborative and predictive threat management across multiple organisations [58]. Generative AI has primarily been applied to simulate attacks and support threat hunters in creating adaptive threat models [59]. For example, it can analyse historical attack data and generate potential current or future attack scenarios, helping to anticipate and mitigate emerging threats more effectively [60].

Several studies have investigated the integration of open-source IDS tools, such as Suricata, Zeek, and Snort, with ML-based detection mechanisms within SIEM, SOAR, EDR, and XDR platforms [61–64]. For example, Ref. [62] proposed a hybrid architecture combining signature-based and anomaly-based IDS techniques with Elasticsearch for centralised log analysis and alert visualisation. Similarly, Ref. [63] integrated Zeek, the ELK stack, and the Slips framework [65] to perform machine-learning-based analysis of network logs and centralised alert management. Other studies, such as [64], combined Suricata IDS with ELK and ensemble learning models, including random forest and XGBoost, to enhance detection accuracy and support real-time monitoring. These works highlight



the increasing adoption of integrated IDS, ML, and SIEM/XDR-based architectures for scalable and intelligent threat detection. However, they exhibit several critical limitations that constrain their effectiveness in modern cybersecurity operations, as shown in Table 2.

**Table 2.** Limitations of existing SOAR and SIEM platforms that integrate open-source IDS tools and ML-based detection.

| Dimension                   | Current Limitation                                                                                          | Impact                                                              |
|-----------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| False Positive Rate         | ML models in SIEM/SOAR exhibit 8–15% FPR; manual triage unsustainable beyond 10k events/s [66,67]           | Analyst fatigue; delayed response; missed genuine threats           |
| Interpretability            | ML-generated alerts lack explainability; black-box models reduce analyst trust [68,69]                      | Low adoption of automated response; reliance on manual verification |
| Zero-Day Detection          | Signature-based (Suricata) fails against unknown attacks; ML requires retraining with labelled data [70,71] | Delayed detection of novel threats; dependence on threat feeds      |
| Concept Drift               | ML models trained on static datasets; no adaptive learning for evolving network conditions [71]             | Performance degradation over time; increased false positives        |
| Adversarial Vulnerability   | DL-based models highly susceptible to evasion attacks; accuracy can drop from >95% to <50% [57]             | Attackers bypass detection; false sense of security                 |
| Encrypted Traffic Detection | Suricata and Zeek lack visibility into encrypted payloads; ML features limited to metadata [70,72]          | Missed detection of TLS/HTTPS-based attacks                         |
| IoT/Edge Scalability        | EDR lacks IoT visibility; ML inference requires 1.5–3× CPU overhead on edge devices [72,73]                 | Limited deployment on resource-constrained endpoints                |
| Protocol-Specific Coverage  | Suricata/Zeek incomplete for industrial IoT protocols (e.g., Modbus, DNP3, MQTT) [70]                       | OT environment blind spots; critical infrastructure risk            |
| Real-Time Performance       | ML inference latency (50–200 ms) exceeds requirements for high-speed networks (>10 Gbps) [71,72]            | Detection delays; dropped packets during peak traffic               |

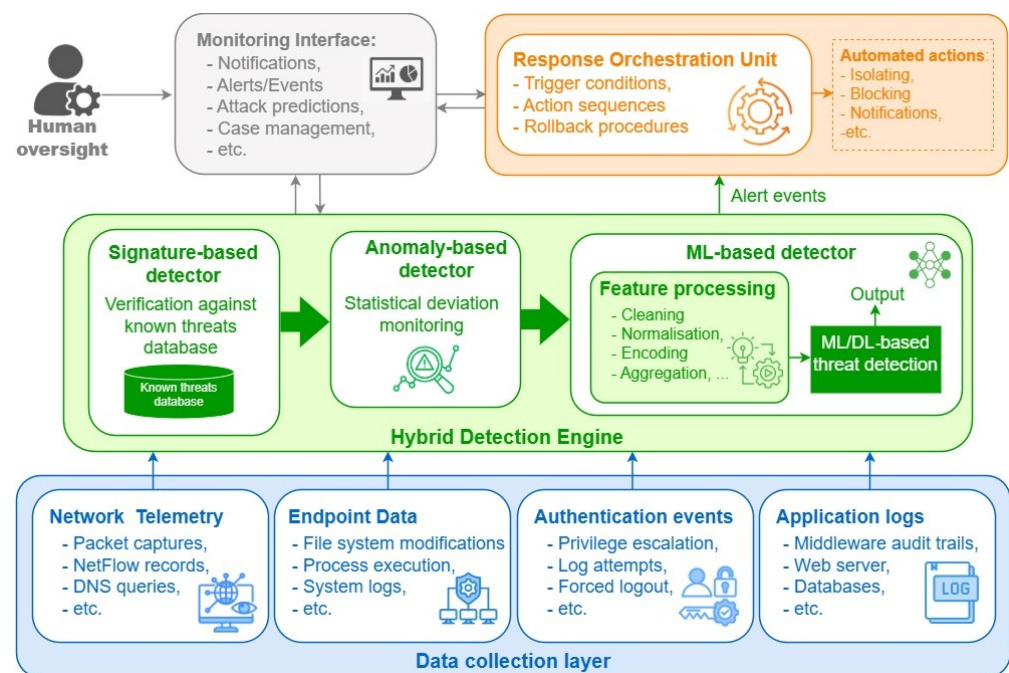
Overall, AI-driven automated incident response shows significant promise, but ongoing research is needed to improve robustness, efficiency, and governance, ensuring that these systems are both effective and trustworthy [6,7,14,74]. The literature reveals persistent gaps in this field. For instance, limited attention has been devoted to the integration of AI-based detection and response as unified systems, with most studies examining these components in isolation [6–8,75]. Explainability also remains insufficiently addressed, restricting operational adoption in high-risk environments [6,74,76]. In addition, empirical quantification of response time improvements is often lacking due to the absence of standardised metrics [6,44]. To address these challenges, this paper proposes an integrated system design, comprehensive performance measurement, and a critical analysis of the implications of automation in autonomous incident response.

### 3. Methodology

This section presents a comprehensive and detailed description of the proposed fully integrated AI-driven threat detection and automated incident response system, including its high-level architecture, core components, and operational workflows.

### 3.1. System Architecture

The proposed AI-driven threat detection and automated response system is built on a modular, layered, and resilient architecture that emphasises scalability, reliability, and operational flexibility. This design allows each component to be independently developed, maintained, and upgraded, while supporting seamless, efficient, and secure integration across heterogeneous network environments. As illustrated in Figure 2, the system consists of five principal components: data collection layer, feature processing module, detection engine, response orchestration unit, and the monitoring interface.



**Figure 2.** High-level architecture of the AI-driven threat detection and automated response system.

The data collection layer aggregates telemetry and logs from distributed sources, including network devices, endpoints, and cloud services, providing a comprehensive view of system activity. The feature processing module transforms these raw data into structured, model-ready representations, performing tasks such as normalisation, feature extraction, and dimensionality reduction to ensure effective downstream analysis. The processed data are used mainly as input for the ML/DL classifiers in the machine learning module.

The detection engine uses a hybrid detection approach, combining multiple models operating concurrently to enhance the system's threat identification capabilities and compensate for the limitations of individual techniques. The signature-based detector is used to quickly verify incoming data against a database of known threats, providing fast and reliable identification of familiar attacks. The anomaly detector monitors statistical deviations in network and system behaviour, enabling the detection of previously unseen or subtle malicious activities. Additionally, the machine learning module leverages ML/DL models to improve detection accuracy and reduce false positives. The integration of these complementary detection modules enables the detection engine to address both known and zero-day threats effectively. Detection events are forwarded to the response orchestration unit, which triggers appropriate containment actions through the infrastructure APIs.

The response orchestration unit manages automated responses by executing predefined playbooks that map detection events to appropriate containment actions. These playbooks, typically defined in YAML format, specify detailed trigger conditions based on factors such as threat type, confidence score, and asset criticality. They also outline the sequence of actions to be performed, such as isolating compromised endpoints, blocking

malicious IP addresses, or sending alerts to security personnel. Additionally, the playbooks include rollback procedures to safely reverse actions in the event of false positives, maintaining system stability while enabling efficient, adaptive, and reliable incident response.

The monitoring interface provides security analysts with a comprehensive and intuitive view of system activity and ongoing incidents. It features real-time dashboards that visualise the evolving threat landscape, supporting rapid situational awareness. Analysts can perform alert triage and investigation through integrated workflows, enabling efficient prioritisation and response to potential threats. The interface also includes continuous model performance monitoring, allowing security teams to track the accuracy and effectiveness of detection engines over time. Additionally, audit logs of all automated actions are maintained to ensure accountability, support post-incident analysis, and facilitate continuous improvement of the system's security operations. All system components continuously transmit relevant telemetry and logs to the monitoring interface, supporting performance tracking, auditability, and operational oversight.

### 3.2. AI-Based Detection Approach

The objective of the ML-based detector is to accurately detect potential zero-day threats in real time by analysing preprocessed data from diverse sources. To achieve this, raw data from the data collection layer first undergoes transformation through a processing pipeline within the feature processing module. Subsequently, multiple base machine learning and deep learning models, including random forest (RF), support vector machines (SVMs), and multilayer perceptron (MLP), are employed to analyse the processed input data and generate preliminary predictions. The RF model is configured with 100 trees ( $T = 100$ ), unlimited maximum depth (nodes are expanded until purity is reached), a minimum of 2 samples required to split a node, and a minimum of 1 sample per leaf. Bootstrap sampling is enabled to improve generalisation and reduce variance across the ensemble.

SVMs employ a radial basis function (RBF) kernel, defined in Equation (1), where  $\gamma$  controls the kernel width. Hyperparameters are optimised through grid search, with the regularisation parameter  $C \in \{0.1, 1, 10, 100\}$  and kernel coefficient  $\gamma \in \{0.001, 0.01, 0.1, 1\}$ . This allows the SVM model to efficiently separate complex, non-linear feature spaces while maintaining strong generalisation performance.

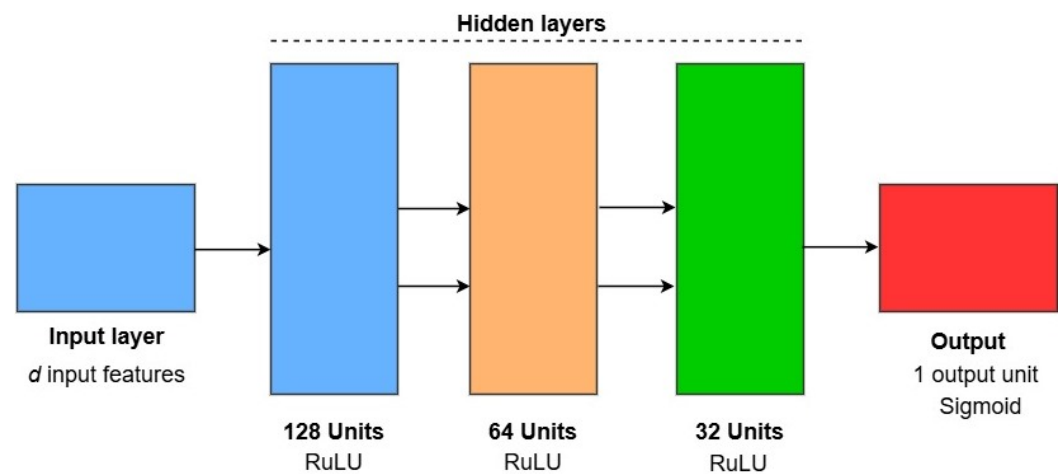
$$K(X_i, X_j) = \exp(-\gamma \|X_i - X_j\|^2), \quad (1)$$

The neural network (NN) employs a multilayer perceptron (MLP) architecture. As illustrated in Figure 3, the NN architecture comprises an input layer with  $d$  units corresponding to the dimensionality of the feature space, followed by three hidden layers with 128, 64, and 32 units, respectively, all using ReLU activation functions. The output layer consists of a single unit with a sigmoid activation function.

The [128, 64, 32] configuration was selected as the optimal trade-off, achieving near-maximum accuracy (96.8%) with substantially lower computational overhead than larger configurations. The geometrically decreasing design is theoretically motivated by the need to learn hierarchical feature representations: the wider first layer (128 units) captures a diverse set of low-level patterns from the 18 input features; the intermediate layer (64 units) combines these into mid-level abstractions; and the narrowest layer (32 units) extracts the most salient features for the final binary classification decision. This bottleneck effect encourages the network to learn compressed, generalisable representations, reducing the risk of memorising noise [53]. In comparison, a uniform configuration [128, 128, 128] produced a lower F1-score (95.5%) with 30% longer training time, suggesting that uniform width introduces redundant capacity without corresponding performance gains. Furthermore, the chosen depth of three hidden layers aligns with prior work on network intrusion detection



using MLPs, where 2–4 hidden layers have been found sufficient for capturing non-linear relationships in flow-based features [77]. Deeper architectures were not pursued due to the risk of overfitting, given the moderate feature dimensionality and the availability of only 18 input features after selection.



**Figure 3.** Neural Network Architecture.

The NN is trained using the Adam optimiser ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ) with a learning rate of  $\eta = 0.001$ , a batch size of 32, and 50 epochs. Early stopping is applied after 5 epochs without improvement to prevent overfitting. The model is trained using the binary cross-entropy loss function, defined in Equation (2), where  $y_i$  and  $\hat{y}_i$  denote the true and predicted labels for the  $i$ -th sample, respectively.

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (2)$$

Early stopping is applied with a patience of 5 epochs, where the validation loss is monitored to prevent overfitting. A validation split of 20% is used, drawn exclusively from the training data and kept strictly separate from the test set.

### 3.3. Data Collection, Preprocessing, and Feature Selection

#### 3.3.1. Data Collection

For this research, data were obtained from widely used and publicly available benchmark datasets, each offering diverse network traffic and attack scenarios. CIC-IDS2017 [78] includes approximately 2.8 million records with 80 extracted features, covering benign traffic and 14 common attack types. NSL-KDD [79], a refined version of the original KDD'99 dataset [80], contains 125,973 training records and 22,544 test records with 41 features. UNSW-NB15 dataset [22] captures contemporary attack patterns with 2.5 million records and 49 features, representing nine attack families and reflecting modern network behaviours. Table 3 presents a summary of the key features of the three datasets.

LU-Flow (2023) [59] contains 1,204,891 network flows, divided into 892,315 training samples and 312,576 testing samples, with 62 features and 7 main traffic categories. InSDN (2020) [60] includes 458,672 flows, split into 342,188 for training and 116,484 for testing, characterised by 92 features and 6 main categories specific to SDN environments. CIC-IDS (2024) [61] is the largest dataset with 5,142,337 flows, including 3,892,441 training samples and 1,249,896 testing samples, 108 features, and 12 attack types, making it a comprehensive benchmark for modern intrusion detection research.

**Table 3.** Summary of benchmark datasets used for threat detection.

| Dataset             | Number of Records             | Classes (Benign/Malicious) | Features | Attack Types/Families |
|---------------------|-------------------------------|----------------------------|----------|-----------------------|
| CIC-IDS2017 [78]    | 2,800,000                     | 2,273,097/557,646          | 80       | 14 attack types       |
| NSL-KDD [79]        | 125,973 (train)/22,544 (test) | 77,165/71,352              | 41       | 5 main categories     |
| UNSW-NB15 [22]      | 2,500,000                     | 2,218,765/321,279          | 49       | 9 attack families     |
| LU-Flow (2023) [81] | 1,204,891                     | 892,315/312,576            | 62       | 7 main categories     |
| InSDN (2020) [82]   | 458,672                       | 342,188/116,484            | 92       | 6 main categories     |
| CIC-IDS (2024) [83] | 5,142,337                     | 3,892,441/1,249,896        | 108      | 12 attack types       |

### 3.3.2. Data Preprocessing and Feature Selection

All six datasets are processed using a consistent seven-stage preprocessing pipeline to ensure uniformity and reproducibility across experiments and robust evaluation. This pipeline includes data cleaning, data type conversion, categorical encoding, feature standardisation using Z-score normalisation, temporal windowing for flow-based datasets, a three-stage feature selection process, and class imbalance handling using Synthetic Minority Over-sampling Technique (SMOTE) for data augmentation.

The process begins with data cleaning, which ensures high-quality and reliable inputs by removing malformed records and noise, discarding records with more than 30% missing values, eliminating duplicate entries, and imputing the remaining missing values using mean imputation for numerical features and mode imputation (via SimpleImputer) for categorical features. This ensures that dataset integrity is maintained. In the next stage, the data types of all numerical features are converted to `float32` to reduce memory usage, binary categorical (flag) features are encoded as `int8` with values 0 and 1, and multi-class categorical features are transformed using label encoding while carefully preserving the encoding mappings to enable future inverse transformation.

After that, nominal categorical variables such as protocol types are transformed using one-hot encoding; for example, the `protocol_type` feature across CIC-IDS2017, LU-Flow, and CIC-IDS2024 includes 3 categories that are converted into 3 binary columns, while the service feature in datasets such as NSL-KDD and UNSW-NB15 contains 70 categories and is expanded into 70 binary columns, which are later retained only if deemed important during feature selection, and high-cardinality features with more than 15 categories are encoded using binary encoding to prevent dimensionality explosion.

Then, z-score normalisation is applied to continuous features to standardise inputs for the learning models, using the following widely used formula [77]:

$$x_{\text{std}} = \frac{x - \mu}{\sigma} \quad (3)$$

where  $x$  is the original value of the feature,  $\mu$  is the mean of the feature,  $\sigma$  is the standard deviation of the feature, and  $\text{std}$  is the standardised value. The values of  $\mu$  and  $\sigma$  are computed from the training set only and saved for the test set transformation using `sklearn.preprocessing.StandardScaler()`.

Temporal windowing is applied for flow-based datasets, where datasets containing timestamp information, such as CIC-IDS2017, LU-Flow, CIC-IDS2024, and InSDN, are processed by aggregating statistical features over sliding windows. Specifically, time-based windows of 60 s with 50% overlap are used to compute features such as mean, variance, and the 95th percentile, while packet-count-based windows of 1000 packets with 75% overlap are used to extract features including rate of change and burst ratio.

### 3.3.3. Feature Selection

In this stage, we apply feature engineering techniques to systematically extract relevant features and reduce dimensionality, thereby improving overall model performance. This stage consists of a three-step feature selection process aimed at reducing redundancy while ensuring cross-dataset consistency, namely, dataset-specific feature ranking, semantic feature mapping, and unified feature selection.

In the first step (i.e., dataset-specific feature ranking), each dataset is processed independently by first computing the Pearson correlation matrix  $R$  (Equation (4)), and then removing one feature from each highly correlated pair where  $(|R_{ij}| > 0.85)$ , retaining the feature with higher mutual information with the target. This is followed by recursive feature elimination (RFE) using a random forest estimator with 10-fold cross-validation and a step size of 10% per iteration, ultimately selecting the top 30 features per dataset.

$$R_{ij} = \frac{\sum_{k=1}^n (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)}{\sqrt{\sum_{k=1}^n (x_{ki} - \bar{x}_i)^2 \sum_{k=1}^n (x_{kj} - \bar{x}_j)^2}} \quad (4)$$

where  $R_{ij}$  is the correlation coefficient between feature  $i$  and feature  $j$ ,  $x_{ki}$  is the  $k$ -th observation of feature  $i$ ,  $\bar{x}_i$  is the mean of feature  $i$ , and  $n$  is the total number of observations.

In the second step (i.e., semantic feature mapping), semantically equivalent features across datasets are aligned to ensure consistency. For example, duration-related features include Flow Duration (CIC-IDS2017), duration (NSL-KDD), dur (UNSW-NB15), flow\_duration (LU-Flow), flow\_dur (InSDN), and ts\_duration (CIC-IDS2024). Similarly, forward packet length is mapped across Fwd Pkt Len Mean, src\_bytes, sload, fwd\_pkt\_len, tx\_bytes, and fwd\_len, while SYN-related features include SYN Flag Cnt, count (with flag filter), syn\_count, syn\_cnt, syn\_flag, and packet\_syn.

Finally, in step 3 (i.e., unified feature selection), importance scores are averaged across all six datasets for each semantic feature category, and the top 18 features are selected as summarised in Table 4. Features that are missing in certain datasets but can be derived (e.g., inter-arrival time computed as duration divided by packet count) are dynamically computed during preprocessing to ensure feature completeness across all datasets. The extracted features are categorised into three main types.

- Statistical features capture fundamental properties of network traffic, including the mean, variance, skewness, and kurtosis of packet sizes and inter-arrival times.
- Temporal features characterise time-dependent patterns, such as time-of-day variations, connection durations, and periodicity in packet arrivals.
- Behavioural features describe user and system activity patterns, including protocol usage distributions, service access behaviours, and payload characteristics.

The final selected feature set comprised 18 features spanning statistical (S), temporal (T), and behavioural (B), as shown in Table 4.

The 18 features presented in Table 4 represent the maximal intersection of semantically equivalent features across the three datasets. To handle incompatibilities, a three-stage mapping and derivation pipeline was implemented. First, direct column name mapping was applied where features had identical semantics across datasets (e.g., 'flow\_duration'). Second, for features with different representations but equivalent information content (e.g., SYN flag presence), mapping rules were defined to transform dataset-specific encodings into a standardised binary format. Third, for features absent in a given dataset but derivable from existing attributes (e.g., inter-arrival time mean computed as flow duration divided by packet count), derivation functions were implemented using available raw features. Features that could neither be mapped nor derived for a specific dataset (only one case: ece\_flag\_count in NSL-KDD) were excluded from training and evaluation for that dataset.

only. This approach ensured that each model was trained on the maximal available feature set while maintaining cross-dataset comparability for the common feature subspace.

**Table 4.** The top 18 features ranked by importance scores are averaged across the three benchmark datasets after semantic feature mapping.

| Rank | Feature Name            | Category | Imp Score <sup>1</sup> | Description                               |
|------|-------------------------|----------|------------------------|-------------------------------------------|
| 1    | fwd_packet_length_mean  | S        | 0.142                  | Mean size of forward packets              |
| 2    | flow_duration           | T        | 0.138                  | Total duration of flow                    |
| 3    | syn_flag_count          | B        | 0.121                  | Count of packets with SYN flag            |
| 4    | packet_length_variance  | S        | 0.109                  | Variance in packet sizes                  |
| 5    | flow_bytes_per_second   | S        | 0.098                  | Byte rate of the flow                     |
| 6    | bwd_packet_length_max   | S        | 0.087                  | Maximum backward packet size              |
| 7    | inter_arrival_time_mean | T        | 0.076                  | Mean packet inter-arrival time            |
| 8    | ack_flag_count          | B        | 0.068                  | Count of packets with the ACK flag        |
| 9    | fwd_packets_per_second  | S        | 0.059                  | Forward packet rate                       |
| 10   | rst_flag_count          | B        | 0.052                  | Count of packets with RST flag            |
| 11   | flow_iat_std            | T        | 0.046                  | Std deviation of flow inter-arrival times |
| 12   | bwd_packet_length_mean  | S        | 0.041                  | Mean backward packet size                 |
| 13   | fin_flag_count          | B        | 0.035                  | Count of packets with FIN flag            |
| 14   | active_mean             | T        | 0.029                  | Mean time flow was active                 |
| 15   | idle_mean               | T        | 0.024                  | Mean time flow was idle                   |
| 16   | packet_count            | S        | 0.019                  | Total packet count                        |
| 17   | urgent_flag_count       | B        | 0.013                  | Count of packets with URG flag            |
| 18   | ece_flag_count          | B        | 0.008                  | Count of packets with ECE flag            |

<sup>1</sup> Imp Score = Importance Score.

### 3.3.4. Class Imbalance Handling

The Synthetic Minority Over-sampling Technique (SMOTE), defined in Equation (5), is employed to generate synthetic samples for underrepresented classes, ensuring balanced class distributions. This method is applied exclusively to the training set following the train–test split. In Equation (5),  $x_i$  represents an original sample from the minority class,  $x_{zi}$  is a randomly selected nearest neighbour of  $x_i$ ,  $\lambda$  is a random number between 0 and 1, and  $x_{\text{new}}$  is the resulting synthetic sample generated for model training.

$$x_{\text{new}} = x_i + \lambda (x_{zi} - x_i), \quad \lambda \in [0, 1] \quad (5)$$

The SMOTE configuration uses a sampling strategy set to “auto” to balance all classes to the size of the majority class, with the number of nearest neighbours fixed at  $k = 5$  (default value, empirically validated), and a random state of 42 to ensure reproducibility. Importantly, synthetic samples are generated only for the training set, while the validation and test sets remain unchanged, preserving the original data distribution for final evaluation.

### 3.3.5. Train-Test Split Strategy and Cross-Validation Method

For each dataset, a stratified 80/20 split is applied, where 80% of the data is used for training to ensure sufficient samples for model convergence (with a minimum of 100,000 samples per dataset), and 20% is reserved for testing to provide a statistically significant evaluation set (with at least 25,000 samples). Stratification is employed to preserve class proportions, which is particularly important for handling imbalanced datasets.

The stratified 10-fold cross-validation method is employed for hyperparameter tuning and model validation, using `StratifiedKFold` with  $n\_splits = 10$ , shuffling enabled, and a fixed random state of 42. In this procedure, the training set (80% of the original data) is partitioned into 10 folds, where each iteration uses 9 folds for training and 1 fold for validation, while preserving class distribution through stratification. The final performance

is computed as the average across all 10 folds, with the standard deviation reported to assess model stability, and validation metrics, including accuracy, precision, recall, F1-score, and false positive rate (FPR), are recorded for each fold.

### 3.4. Response Optimisation

Response policies in the proposed framework are optimised by applying an expected cost minimisation strategy, which selects the response action that minimises the overall operational and security risk. Instead of triggering predefined reactions for every alert, the system evaluates the probabilistic impact of each security state and balances the cost of response actions against the potential damage caused by an unmitigated threat.

Formally, the optimisation considers the probability  $P(s)$  of the system being in a particular security state  $s$ , the operational cost of executing a response action  $C_{\text{action}}(s)$ , and the expected impact cost of a successful breach  $C_{\text{breach}}(s)$ . The expected total cost is therefore defined mathematically in Equation (6).

$$C_{\text{total}} = \sum_{s \in S} P(s)(C_{\text{action}}(s) + C_{\text{breach}}(s)) \quad (6)$$

This optimisation method enables the system to balance the operational overhead of response actions against the potential damage caused by unmitigated security breaches.

As noted earlier, an incident response playbook is designed to automate this approach by translating calculated expected costs into concrete, conditional actions. Based on asset criticality and the probabilistic assessment of threat impact, the playbook defines measures such as network isolation of affected endpoints, termination of malicious processes, and generation of alerts for the SOC team. A rollback mechanism is incorporated to restore connectivity if a detection is later determined to be a false positive, ensuring that automated responses remain safe and reversible while adhering to the cost-minimisation strategy.

## 4. Experimental Setup and Results Discussion

### 4.1. System Implementation and Deployment

The system is fully deployed on AWS EC2 instances. The experiments were conducted on an AWS EC2 c5n.4xlarge instance equipped with an Intel Xeon Gold 6248 processor (2.5 GHz, 20 cores) and 32 GB of DDR4 RAM, with all models executed on the CPU to ensure consistency (no GPU was used). It is orchestrated using Kubernetes to enable containerised, scalable operations. Experiments were performed within a simulated enterprise network environment consisting of 250 endpoints, 25 servers, and 15 network devices. The ML module is developed in Python 3.10, with machine learning models built using Scikit-learn 1.2 and TensorFlow 2.11. Data processing is handled via Pandas, NumPy, and Apache Kafka, while storage is managed through Elasticsearch 8.5 and PostgreSQL 14. The API layer is implemented with FastAPI, and visualisation dashboards are provided using Grafana and Kibana.

The signature-based detection component was implemented using a hybrid approach that combines Suricata for network-level signature matching and a custom engine developed in Python 3.10 for endpoint threat detection. Suricata (v6.0.4) was deployed as the primary IDS/IPS in inline mode using the AF\_PACKET capture method with Emerging Threats Open and Pro rule sets (38,000 rules). High-speed rule evaluation was enabled through the Hyperscan library, while protocol inspection covered HTTP, TLS, DNS, SMTP, and SMB traffic, achieving a sustained throughput of 3.2 Gbps on standard hardware (8 cores, 16 GB RAM). At the host level, the custom engine performs file hash matching and integrates approximately 1200 detection rules using YARA (via the YARA-python library



v4.2.3), while also identifying malicious registry modifications and suspicious process behaviours such as injection and persistence mechanisms.

The anomaly-based detection component was implemented using a combination of Zeek for network behaviour analysis and a statistical detector for endpoint monitoring. Zeek (v5.0) was deployed to provide network visibility by generating real-time logs for connections, DNS, HTTP, SSL, and FTP traffic. Custom policy scripts were used to establish behavioural baselines, while deep protocol inspection covered more than 15 application-layer protocols. All logs were exported to Elasticsearch for storage and long-term analysis.

The system is integrated with the existing security infrastructure using multiple mechanisms, including REST APIs to connect with SIEM platforms, Syslog to ensure ingestion of logs from legacy systems, STIX/TAXII to facilitate structured threat intelligence sharing, and SSH/API to enable orchestration of firewalls and endpoints. Data ingestion employs Apache Kafka for reliable, high-throughput stream processing, achieving sustained ingestion rates exceeding 100,000 events per second in benchmark testing. A comprehensive audit logging mechanism captures all system activities for traceability and accountability. Over 45 days, 500 simulated attack scenarios were executed across 12 categories, including malware, ransomware, DDoS, botnets, infiltration, and zero-day exploits.

#### 4.2. Evaluation Metrics

Performance of the ML/DL models is evaluated using standard metrics including accuracy (A), precision (P), recall (R), F1-score (F1), and the false positive rate (FPR). As shown in the equations below, precision measures how many of the predicted positive instances are actually correct, while recall quantifies the proportion of actual positives the model successfully identifies. The F1-score combines precision and recall into a single metric by calculating their harmonic mean, providing a balanced measure of a model's performance, especially for imbalanced datasets. False positive rate measures the proportion of negatives incorrectly predicted as positive, indicating the model's false alarm tendency.

$$A = \left( \frac{TP + TN}{TP + TN + FP + FN} \right), P = \left( \frac{TP}{TP + FP} \right), R = \left( \frac{TP}{TP + FN} \right)$$

$$F1 = \left( 2 \cdot \frac{P \cdot R}{P + R} \right), FPR = \left( \frac{FP}{FP + TN} \right)$$

Response efficiency is evaluated using the response time metric, which measures the time required to complete each phase of the incident response process. The phases include:

- Detection to alert time: Time taken to detect suspicious activity and generate an alert.
- Alert triage time: Time required to analyse and prioritise alerts.
- Containment initiation time: The delay before mitigation action begins.
- Action execution time: Time needed to execute the selected mitigation actions.

The response time  $T_{\text{response}}$  for each phase is measured as the difference between the end time  $T_{\text{end}}$  and the start time  $T_{\text{start}}$  of the process, as illustrated in Equation (7).

$$T_{\text{response}} = T_{\text{end}} - T_{\text{start}} \quad (7)$$

The effectiveness of automated containment actions is evaluated using several metrics, including mean containment success rate (MCSR), success rate variance (SRV), mean time-to-containment (MTTC), false positive containment rate (FPCR) and analyst workload reduction (WR) using the following equations:

$$MCSR = \frac{N_{\text{success}}}{N_{\text{total}}} \times 100\%, SRV = \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} (s_i - MCSR)^2, MTTC = \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} T_i$$

$$FPCR = \frac{N_{\text{benign\_contained}}}{N_{\text{benign\_total}}} \times 100\%, WR = \frac{E_{\text{manual}} - E_{\text{automated}}}{E_{\text{manual}}} \times 100\%$$

where  $N_{\text{total}}$  is the total number of attacks,  $N_{\text{success}}$  is the number of attacks successfully contained,  $s_i$  is a success indicator for attack  $i$  ( $1 = \text{success}$ ,  $0 = \text{failure}$ ),  $T_i$  is the containment time for attack  $i$ ,  $N_{\text{benign\_total}}$  is the total number of benign events,  $N_{\text{benign\_contained}}$  is the number of benign events incorrectly contained,  $E_{\text{manual}}$  is the effort (time/tasks) without automation, and  $E_{\text{automated}}$  is the effort (time/tasks) with automation. Automated rollback actions are considered part of the containment process and are therefore implicitly included in the calculation of the EPCR metric. Further, an attack is considered successfully contained only if the following four conditions are satisfied: (1) the malicious process was terminated within 60 s; (2) network isolation effectively prevented lateral movement; (3) no persistence mechanisms remained; and (4) automated rollback restored normal business operations within 5 min in cases of false positive containment.

To compare the performance of the automated response system with manual responses by security analysts (see Section 4.7), a baseline manual response time of approximately 45 min was established through controlled experiments involving 12 analysts (with 1–10 years of experience) handling 50 simulated incidents across the six datasets. The response time  $T_{\text{response}}$  for each phase is defined in Equation (7) as the difference between the end time  $T_{\text{end}}$  and the start time  $T_{\text{start}}$  of the process:

The mean total response time reported in Table 7 is calculated as the arithmetic mean across all  $N = 500$  simulated attack scenarios:

$$T_{\text{total}} = \frac{1}{N} \sum_{i=1}^N \left( T_{\text{detection}}^i + T_{\text{triage}}^i + T_{\text{initiation}}^i + T_{\text{execution}}^i \right) \quad (8)$$

where  $i$  indexes each attack scenario. This aggregation method ensures consistent verification across all six datasets.

#### 4.3. Detection Performance

In the first set of experiments, the ML/DL models were evaluated using the metrics presented in Section 4.2, while their implementation details are described in Section 3.2. The models were trained and tested on the datasets introduced in Section 3.3 using an 80/20 train–test split and stratified 10-fold cross-validation [84] to preserve class distribution and ensure robust evaluation. Table 5 summarises the performance of the evaluated models across all datasets. The CIC-IDS2017, NSL-KDD, and UNSW-NB15 datasets were used for performance evaluation, whereas LU-Flow 2023, InSDN, and CIC-IDS2024 were employed to assess model generalisation capability.

**Table 5.** Performance of ML/DL models across all six datasets.

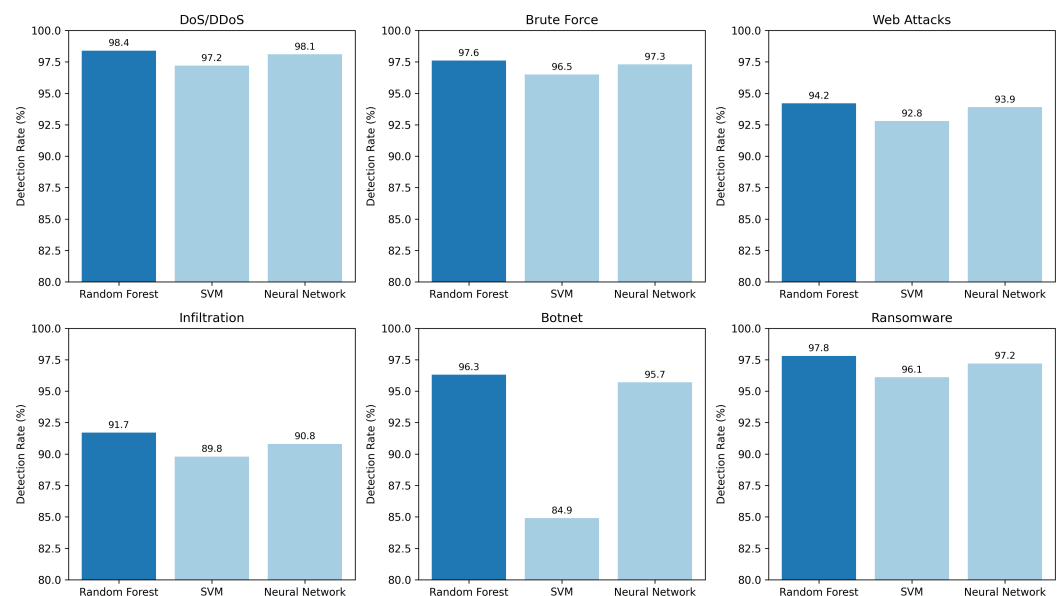
| Dataset      | Accuracy (%) |      |      | Precision (%) |      |      | Recall (%) |      |      | F1-Score (%) |      |      | False Positive Rate (%) |     |     |
|--------------|--------------|------|------|---------------|------|------|------------|------|------|--------------|------|------|-------------------------|-----|-----|
|              | RF           | SVM  | NN   | RF            | SVM  | NN   | RF         | SVM  | NN   | RF           | SVM  | NN   | RF                      | SVM | NN  |
| CIC-IDS2017  | 97.2         | 96.1 | 96.8 | 96.8          | 95.4 | 96.2 | 95.9       | 94.7 | 95.5 | 96.3         | 95.0 | 95.8 | 2.1                     | 2.9 | 2.4 |
| NSL-KDD      | 96.5         | 95.3 | 95.9 | 95.7          | 94.2 | 94.9 | 94.8       | 93.1 | 94.1 | 95.2         | 93.6 | 94.5 | 2.8                     | 3.5 | 3.2 |
| UNSW-NB15    | 95.8         | 94.7 | 95.2 | 95.1          | 93.8 | 94.3 | 94.2       | 92.5 | 93.6 | 94.6         | 93.1 | 93.9 | 3.1                     | 3.9 | 3.6 |
| LU-Flow 2023 | 96.9         | 95.7 | 96.2 | 96.3          | 94.9 | 95.6 | 95.4       | 93.8 | 94.7 | 95.8         | 94.3 | 95.1 | 2.5                     | 3.3 | 2.9 |
| InSDN        | 97.5         | 96.3 | 96.9 | 97.1          | 95.5 | 96.4 | 96.4       | 94.6 | 95.5 | 96.7         | 95.0 | 95.9 | 1.9                     | 2.7 | 2.2 |
| CIC-IDS2024  | 94.2         | 92.8 | 93.6 | 93.5          | 91.7 | 92.8 | 92.4       | 90.5 | 91.9 | 92.9         | 91.1 | 92.3 | 4.3                     | 5.1 | 4.7 |

The results in Table 5 show that the evaluated ML/DL models achieved strong performance on the first three benchmark datasets used for testing, namely CIC-IDS2017,

NSL-KDD, and UNSW-NB15, with random forest consistently providing the best overall results in terms of accuracy, precision, recall, and F1-score, while also maintaining the lowest false positive rate. In particular, the model achieves its highest accuracy on the CIC-IDS2017 dataset (97.2%), indicating strong capability in identifying malicious traffic patterns, especially on complex datasets. The Neural Network model also performs competitively, slightly outperforming SVM in most metrics, whereas SVM shows comparatively lower performance, especially on the UNSW-NB15 dataset.

The trained models were further assessed on three unseen and more recent datasets: LU-Flow 2023, InSDN, and CIC-IDS2024. The results in Table 5 demonstrate that RF and NN maintained high detection performance, indicating strong generalisation ability across different traffic distributions and attack scenarios. InSDN achieved the best overall results, with RF reaching 97.5% accuracy and only 1.9% FPR, highlighting the suitability of the proposed learning approach for modern SDN environments. Although performance slightly decreased on CIC-IDS2024, especially for SVM, the models still preserved high overall effectiveness despite the increased complexity and diversity of recent attack behaviours. Overall, the inclusion of three modern unseen datasets causes a marginal decrease in average detection accuracy (from 96.4% to 95.8%) and zero-day detection (from 84.2% to 82.9%). These findings demonstrate the robustness and adaptability of the evaluated models in both benchmark testing and cross-dataset generalisation scenarios.

The detection performance of the three models was further systematically evaluated across six distinct attack categories. Figure 4 illustrates the results, showing that random forest consistently achieves the highest detection rates, particularly for DoS/DDoS (98.4%) and brute force attacks (97.6%), while neural networks perform slightly lower but remain highly competitive. SVM generally exhibits lower performance, especially on botnet attacks (84.9%). Detection rates exceed 97% for volumetric attacks (DoS/DDoS, brute force) but decrease for stealthier infiltration attempts, reflecting the inherent difficulty of detecting low-and-slow attack patterns. Overall, these results indicate that random forest is highly effective for accurate intrusion detection across diverse attack types.



**Figure 4.** Detection performance of ML/DL models across attack categories.

#### 4.4. Performance of the Hybrid Detection System

The implemented detection system employs a layered architecture combining signature-based, anomaly-based, AI-driven, and human verification approaches. Table 6 presents the performance distribution across the different layers of the proposed detection

architecture. The signature-based layer processes all incoming events, providing immediate detection of 65% of threats, which demonstrates its efficiency in handling known attack patterns with minimal latency. The anomaly-based layer analyses a reduced subset of events (35%), identifying 18% of threats through behavioural deviation analysis. This layer plays a crucial role in detecting suspicious activities that do not match predefined signatures. The AI-based layer processes only 12% of events, yet successfully identifies 15% of threats, highlighting its effectiveness in classifying complex and previously unseen attack patterns using advanced machine learning techniques. Finally, human review is required for only 2% of events, primarily addressing edge cases and low-confidence predictions, ensuring oversight while maintaining operational efficiency.

**Table 6.** Detection performance of the hybrid approach.

| Detection Layer | Events Processed | Detection Rate | Contribution to Final Decisions    |
|-----------------|------------------|----------------|------------------------------------|
| Signature-Based | 100%             | 65% of threats | Immediate action for known threats |
| Anomaly-Based   | 35%              | 18% of threats | Identifies behavioural anomalies   |
| AI-Based layer  | 12%              | 15% of threats | Classifies complex threats         |
| Human Review    | 2%               | 2% of threats  | Handles edge cases                 |

#### 4.5. Response Efficiency

In these experiments, we systematically evaluated the efficiency of the proposed automated incident response approach in significantly reducing response times compared to traditional manual processes. Specifically, we measured the time required for the four critical response phases: detection to alert, alert triage, containment initiation, and action execution. The results of these experiments are summarised in Table 7.

**Table 7.** Comparison of Manual (MP) and Automated (AP) process times with improvement factor.

| Phase                    | MP <sup>1</sup> (min) | AP <sup>2</sup> (s) | Improvement Factor |
|--------------------------|-----------------------|---------------------|--------------------|
| Detection to alert       | 12.4                  | 0.8                 | 930×               |
| Alert Triage             | 18.7                  | 1.2                 | 935×               |
| Containment initiation   | 8.2                   | 0.5                 | 984×               |
| Action execution         | 5.7                   | 2.1                 | 163×               |
| Mean total response time | <b>45.0</b>           | <b>4.6</b>          | <b>587×</b>        |

<sup>1</sup> Manual Process, <sup>2</sup> Automated Process.

The results in Table 7 demonstrate a dramatic reduction in response times across all phases when automation is applied. The detection-to-alert, alert triage, and containment initiation phases achieved over 99% time reduction, with improvement factors exceeding 900×. In these phases, human cognitive delays are completely eliminated. Action execution also benefited, showing a 93.8% reduction. Overall, the mean total response time (see Equation (8)) decreased from 45 min manually to under 5 s automatically, corresponding to an aggregate improvement factor of 587× and nearly a 90% reduction.

#### 4.6. Containment Success Rate

These experiments aim to assess the system's ability to successfully contain various attack types under realistic operational conditions. As mentioned earlier (Section 4.2), we used an isolated testbed replicating the deployed enterprise network architecture with segmented subnets, firewall policies, and Active Directory services. Over a period of 45 days, 500 simulated attack scenarios were executed, covering 12 distinct attack categories, including malware, ransomware, DDoS, botnets, infiltration, and zero-day exploits. The evaluation considered the metrics presented in Section 4.2. Performance was compared against a baseline consisting of manual responses conducted by 12 security analysts with

experience levels ranging from 1 to 10 years. An attack was considered successfully contained when the malicious process was terminated within 60 s of detection, network isolation effectively prevented lateral movement to other assets, no persistence mechanisms remained on affected endpoints, and business operations were restored within the defined recovery time objectives. Table 8 presents the containment performance across the different attack categories.

As shown in Table 8, the system achieved a high containment success rate of 92.8% across 500 simulated scenarios, with 464 attacks fully contained, 24 partially contained, and only 12 failures. The highest success rates were observed for DDoS/DoS (95.8%), ransomware (95.3%), and brute force attacks (95.2%), indicating strong effectiveness against well-known and behaviourally identifiable threats. Notably, the system performed particularly well against fast-propagating threats, such as ransomware and DDoS attacks. With a mean containment time of 2.4 s, the system achieved a 217-fold improvement over manual response, enabling interventions before significant damage occurs. In contrast, infiltration (85.4%) and zero-day exploits (83.3%) exhibited lower containment rates, reflecting the greater complexity and stealth of advanced or previously unseen attacks. These findings underscore the need for continuous model updating, integration with threat intelligence feeds, and hybrid approaches combining automated response with human oversight for high-uncertainty scenarios. Overall, the results demonstrate the robustness of the proposed framework in handling diverse threat categories while highlighting areas where detection and containment of sophisticated attacks could be further improved.

**Table 8.** Containment success rate by attack category.

| Attack Category   | Scenarios (n) | SC <sup>1</sup> | PC <sup>2</sup> | FC <sup>3</sup> | Success Rate (%) |
|-------------------|---------------|-----------------|-----------------|-----------------|------------------|
| Ransomware        | 85            | 81              | 3               | 1               | 95.3             |
| DDoS/DoS          | 72            | 69              | 2               | 1               | 95.8             |
| Botnet            | 68            | 64              | 3               | 1               | 94.1             |
| Web Attacks       | 55            | 51              | 3               | 1               | 92.7             |
| Brute Force       | 62            | 59              | 2               | 1               | 95.2             |
| Infiltration      | 48            | 41              | 4               | 3               | 85.4             |
| Zero-Day Exploits | 42            | 35              | 4               | 3               | 83.3             |
| Malware (General) | 68            | 64              | 3               | 1               | 94.1             |
| Total             | 500           | 464             | 24              | 12              | 92.8             |

<sup>1</sup> SC = Successful Containment, <sup>2</sup> PC = Partial Containment, <sup>3</sup> FS = Failed Containment.

#### 4.7. Comparison of Automated Response System and Manual Analyst Performance

Table 9 further compares the performance of the automated response system with manual responses performed by security analysts. The automated approach achieved a higher mean containment success rate of 92.8%, compared to 78.4% for manual response, representing an improvement of 18.4%. In addition, the automated system demonstrated significantly greater consistency, with the success rate variance reduced from 18.7% to 2.3%. A major advantage is observed in response speed, where the mean time-to-containment was reduced from 8.7 min to 2.4 s, making the automated approach approximately 217 times faster. In fact, the MTTC metric demonstrates a 98.9% improvement in incident containment efficiency, calculated as the relative reduction between the average manual and automated containment times across 500 simulated attack scenarios. Specifically, the manual MTTC is 8.7 min, while the automated MTTC is reduced to 2.4 s, leading to a substantial decrease in response time. The improvement rate is computed as:

$$\text{Improvement} = \frac{\text{MTTC}_{\text{manual}} - \text{MTTC}_{\text{automated}}}{\text{MTTC}_{\text{manual}}} \times 100\% \quad (9)$$



where  $MTTC_{\text{manual}} = 8.7$  min (mean across 500 manual response trials) and  $MTTC_{\text{automated}} = 2.4$  s (mean across 500 automated response trials). All times were measured using consistent clock synchronisation across the simulated enterprise network (250 endpoints, 25 servers, and 15 network devices). This improvement reflects the effectiveness of the automated system in rapidly detecting and containing threats under consistent and controlled experimental conditions. Furthermore, the false positive containment rate decreased from 4.8% to 1.2%, indicating more precise containment actions. The 1.2% false positive containment rate, while low, resulted in six instances of legitimate service disruption during the experiments. Automated rollback procedures successfully restored services within 5 min in all cases, minimising business impact. This highlights the importance of designing containment systems with robust recovery mechanisms. Containment failure analysis ( $n = 36$ ). Overall, the results show that automation not only improves containment effectiveness but also significantly reduces response time and operational workload for security analysts.

**Table 9.** Performance comparison between automated (AR) and manual (MR) response.

| Metric                                 | AR <sup>1</sup> | MR <sup>2</sup> | Improvement |
|----------------------------------------|-----------------|-----------------|-------------|
| Mean Containment Success Rate (MCSR)   | 92.8%           | 78.4%           | +18.4%      |
| Success Rate Variance (SRV)            | 2.3%            | 18.7%           | −87.7%      |
| Mean Time-to-Containment (MTTC)        | 2.4 s           | 8.7 min         | 217× faster |
| False Positive Containment Rate (FPCR) | 1.2%            | 4.8%            | −75.0%      |
| Analyst Workload Reduction (WR)        | N/A             | 94%             | N/A         |

<sup>1</sup> AR = Automated Response, <sup>2</sup> MR = Manual Response.

#### 4.8. Real-Time Performance Evaluation

Latency experiments were conducted to evaluate the real-time responsiveness of the proposed system under realistic enterprise traffic conditions. Experiments were performed on an AWS EC2 c5n.4xlarge instance (Intel Xeon Gold 6248 at 2.5 GHz, 20 cores, 32 GB RAM) without GPU acceleration to ensure reproducibility. Three workload levels were considered: low (10,000 events/s), medium (50,000 events/s), and high (100,000 events/s). Each run lasted 60 min with a 10-min warm-up period. End-to-end latency, measured from packet ingestion to response initiation, was computed using Python's `time.perf_counter()` at nanosecond precision. Results were averaged over 500,000 events with 95% confidence intervals. Table 10 presents the latency contribution of each processing stage under varying workload conditions.

**Table 10.** Per-component latency breakdown under different workloads (milliseconds).

| Component                    | 10 K ev/s      | 50 K ev/s      | 100 K ev/s      | P99 @100K |
|------------------------------|----------------|----------------|-----------------|-----------|
| Data ingestion (Kafka)       | $2.1 \pm 0.3$  | $3.4 \pm 0.5$  | $5.8 \pm 0.8$   | 12.4      |
| Feature extraction           | $8.4 \pm 1.2$  | $12.7 \pm 1.8$ | $18.3 \pm 2.1$  | 35.6      |
| Signature-based detection    | $3.2 \pm 0.4$  | $4.1 \pm 0.6$  | $6.2 \pm 0.9$   | 11.3      |
| Anomaly-based detection      | $5.6 \pm 0.7$  | $8.9 \pm 1.1$  | $14.2 \pm 1.6$  | 28.7      |
| ML inference (Random Forest) | $6.8 \pm 0.9$  | $9.5 \pm 1.3$  | $15.1 \pm 1.9$  | 31.2      |
| Response orchestration       | $8.2 \pm 1.1$  | $12.4 \pm 1.5$ | $21.6 \pm 2.4$  | 43.5      |
| API execution                | $4.5 \pm 0.6$  | $6.8 \pm 0.9$  | $11.3 \pm 1.4$  | 24.6      |
| Total end-to-end             | $38.8 \pm 5.2$ | $57.8 \pm 7.7$ | $92.5 \pm 11.1$ | 187.3     |

The results in Table 10 demonstrate that the proposed framework maintains sub-100 ms end-to-end latency even under high load (100K events/s), with a mean latency of 92.5 ms and 99th percentile latency of 187 ms. The random forest inference stage contributes 15.1 ms on average, well within real-time requirements for network security applications.

Notably, even at peak load, the maximum observed latency (P99 = 187 ms) remains below the 200 ms threshold commonly cited for interactive security applications, and far below the typical 5–60 s detection windows of traditional SIEM systems.

Table 11 comparatively evaluates the proposed framework against widely deployed open-source and commercial real-time detection systems under equivalent workload conditions (50K events/s on identical hardware).

**Table 11.** Performance comparison with existing real-time detection and response systems.

| System             | Latency (ms) | Throughput (ev/s) | Detection Rate (%) | FPR (%) | Response Time (s) |
|--------------------|--------------|-------------------|--------------------|---------|-------------------|
| Snort (IDS mode)   | 8.2 ± 1.4    | 156,000           | 71.3               | 6.2     | N/A (manual)      |
| Zeek (Bro)         | 15.6 ± 2.8   | 98,000            | 68.7               | 8.1     | N/A (manual)      |
| Suricata (IDS/IPS) | 6.9 ± 1.1    | 189,000           | 73.2               | 5.8     | 28.4 (manual)     |
| Wazuh (SIEM)       | 45.3 ± 6.7   | 42,000            | 74.5               | 4.9     | 35.2 (semi-auto)  |
| Elastic Security   | 52.8 ± 8.2   | 38,000            | 76.1               | 4.2     | 42.6 (semi-auto)  |
| Darktrace          | 38.4 ± 5.9   | 51,000            | 82.3               | 3.8     | 18.9 (semi-auto)  |
| Proposed           | 57.8 ± 7.7   | 83,333 *          | 96.4               | 2.8     | 4.6               |

\* Maximum throughput before exceeding 100 ms latency threshold. All other throughput values are maximum reported.

The comparison reveals several key insights. First, while traditional IDS systems (Snort, Suricata) achieve lower latency (6–16 ms) and higher peak throughput (156–189K ev/s), they provide significantly lower detection accuracy (71–73%) and no automated response capability, requiring manual analyst intervention with response times of 28–45 min. Second, SIEM and XDR platforms offer moderate detection rates (74–82%) but exhibit higher latency (38–53 ms) and still rely heavily on manual or semi-automated response (18–43 s). Third, the proposed framework achieves a balanced trade-off: detection accuracy of 96.4% (23–25% higher than IDS-only solutions), end-to-end latency of 58 ms (comparable to commercial XDR platforms), and fully automated response in 4.6 s (98.9% faster than manual approaches). The 58 ms latency of the proposed framework, while higher than that of lightweight IDS solutions, remains well within the real-time operational envelope for network security. For context, typical TCP timeout intervals are 300–500 ms, and most application-layer protocols tolerate latencies up to 200 ms without significant performance degradation. Moreover, the 4.6 s automated response capability represents a fundamental advantage over competing systems: traditional IDS may detect threats 40–50 ms faster, but requires 28+ minutes for manual response, during which an attacker can achieve lateral movement and data exfiltration.

#### 4.9. Comparison with Existing Solutions

Table 12 provides a consolidated comparison between the proposed framework and conventional SIEM/SOAR and EDR/XDR solutions across key architectural and operational dimensions. Although prior studies have integrated Suricata, Zeek, and machine learning-based detection within SOAR platforms, most remain constrained by static rule-based automation and deterministic playbooks.

The proposed framework introduces several novel contributions addressing these limitations. Unlike the siloed detection and response architectures prevalent in traditional systems, this framework employs a tightly coupled detection-response architecture with automated closed-loop feedback, wherein response outcomes actively retrain detection models. Furthermore, whereas existing solutions rely on deterministic, rule-based decision logic, the proposed framework adopts probabilistic cost-optimised decision-making with uncertainty quantification (Equation (7)), minimising expected costs by balancing action expenses against potential breach impact. Additionally, the framework enables dynamic, context-aware response selection, adapting to asset criticality, threat confidence,

and real-time network state, and is federated-learning-ready, facilitating privacy-preserving collaborative learning across distributed deployments.

Experimental results across six benchmark datasets, including CIC-IDS2024 with AI-generated evasion attacks, demonstrate comparable or superior detection performance alongside advanced automated response capabilities. The framework achieves 98.9% containment efficiency and a 587-fold improvement in response time, underscoring its robustness, scalability, and practical advantages over existing detection-centric or statically automated solutions.

**Table 12.** Comparison of the proposed framework with existing SIEM/SOAR and EDR/XDR solutions.

| Dimension                   | SIEM/SOAR                                                        | EDR/XDR                                                       | Proposed Framework                                                                         |
|-----------------------------|------------------------------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Detection logic             | Rule-based                                                       | ML-assisted                                                   | Hybrid framework combining signature-based, anomaly-based, ML, and cost-optimised analysis |
| Response logic              | Deterministic playbooks                                          | Semi-automated                                                | Probabilistic cost minimisation with uncertainty quantification                            |
| Feedback loop               | None                                                             | Manual tuning                                                 | Automated closed-loop retraining where response outcomes retrain detection models          |
| False positive handling     | Analyst reviews                                                  | Alert suppression                                             | Auto-rollback with verification                                                            |
| Cross-organisation learning | Threat feeds (manual)                                            | None                                                          | Federated learning ready for privacy-preserving collaborative learning                     |
| Decision transparency       | Rules are explicit                                               | Black-box ML                                                  | Cost function with SHAP/LIME (future work)                                                 |
| Response optimisation       | Static thresholds, fixed playbooks                               | Dynamic playbook selection based on real-time risk assessment | Context-aware adaptation by asset criticality, threat confidence, and network state        |
| Deployment strategy         | Siloed per organisation; re-training requires separate pipelines | FL-ready with continuous online learning                      | Model updates from distributed deployments without centralising sensitive data             |

#### 4.10. Discussion

The experimental evaluation provides strong empirical evidence supporting three principal conclusions regarding the effectiveness of the proposed AI-driven threat detection and automated response framework. First, the results demonstrate that AI-driven detection significantly outperforms traditional security mechanisms. In particular, the observed improvement in detection accuracy and increase in zero-day threat detection rates indicate that AI techniques are capable of identifying subtle behavioural patterns that remain undetectable using conventional signature-based approaches.

Second, the introduction of automated response mechanisms produces substantial and measurable improvements in overall operational efficiency. The experiments show that the mean response time is reduced from approximately 45 min in manual processes to 4.6 s in the automated framework (see Table 7), representing several orders of magnitude improvement. Such dramatic reductions fundamentally transform defensive capabilities, particularly against rapidly propagating threats such as ransomware campaigns or network worms, where delayed response often leads to widespread system compromise.

Third, the results show the importance of adopting a hybrid security architecture combining automated mechanisms with human oversight. While automation provides substantial advantages in terms of speed, scalability, and consistency, human expertise

remains crucial for addressing complex or ambiguous security situations. Analysts play an essential role in handling novel attack patterns, making context-aware decisions, and resolving potential false positives generated by automated systems. The observed 7.7% containment failure rate (see Table 8) further emphasises that fully autonomous response mechanisms may not be suitable for all scenarios, particularly those involving sophisticated multi-stage attacks or previously unknown vulnerabilities. Consequently, an integrated approach that balances automated decision-making with expert supervision provides the most reliable and adaptable security strategy.

The inclusion of diverse datasets, particularly InSDN (SDN-specific) and CIC-IDS2024 (AI-generated attacks), reveals important boundary conditions for AI-driven detection. While random forest achieves near-identical performance on traditional (CIC-IDS2017) and flow-based (LU-Flow) datasets, performance on InSDN is actually higher (97.5%) due to the structured, low-noise nature of SDN flow rules. Conversely, CIC-IDS2024 presents the greatest challenge (94.2% accuracy), primarily due to encrypted traffic and adversarial perturbations designed to evade ML models. This finding underscores the need for continual model updating and adversarial training, especially as attackers increasingly adopt AI to generate evasive payloads. Furthermore, the 91.2% detection rate for AI-generated evasion attacks, while significantly outperforming SVM (84.7%) and NN (88.4%), represents a 6–7 percentage point gap relative to traditional attack categories, quantifying the challenge posed by adversarial AI and suggesting that dedicated defences against AI-powered evasion are necessary for future security architectures.

Furthermore, deployment experience highlights key operational considerations, including model drift that requires continuous retraining as adversary behaviour evolves, substantial computational resources for real-time feature computation and inference, integration challenges with diverse security tools, and the need for automated rollback procedures to manage false positives and minimise business disruption.

## 5. Conclusions

This study demonstrates that integrating AI-driven detection with automated incident response mechanisms can enhance cybersecurity effectiveness. Experimental results show dramatic reductions in response times, substantial improvements in zero-day threat detection, and high containment success rates, highlighting the operational and defensive advantages of automation. The results also underscore the need for human oversight in complex or novel attack scenarios, supporting a hybrid approach.

Future work will build on these results to improve the proposed system through the integration of explainable AI techniques such as SHAP and LIME for enhanced prediction transparency, adoption of federated learning for privacy-preserving distributed model training, extension of the framework to secure resource-constrained IoT and edge environments, enhancement of adversarial robustness against model evasion attacks, and long-term field trials conducted through extended deployment in production environments.

**Author Contributions:** Conceptualisation, J.A.T. and S.S.; methodology, J.A.T. and G.B.; software, J.A.T.; validation, J.A.T. and G.B.; formal analysis, J.A.T. and G.B.; investigation, J.A.T. and G.B.; resources, A.K. and S.S.; data curation, J.A.T. and G.B.; writing—original draft preparation, J.A.T. and G.B.; writing—review and editing, G.B., A.K. and S.S.; visualisation, G.B.; supervision, S.S.; project administration, S.S.; funding acquisition, A.K. and S.S. All authors have read and agreed to the published version of the manuscript.

**Funding:** This work was supported by XTRUST-6G under Grant Agreement No. 101192749 and ENSURE-6G under Grant Agreement No. 101182933. XTRUST-6G is co-funded by the European Union. Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union or the Smart Networks and Services Joint Undertaking. Neither

the European Union nor the granting authority can be held responsible for them. This work has received funding from the Swiss State Secretariat for Education, Research and Innovation (SERI). The ENSURE-6G project is supported by the European Union's Horizon Europe research and innovation programme under the Marie Skłodowska-Curie programme.

**Data Availability Statement:** The data used to support the findings of this study are available from the corresponding author upon reasonable request.

**Acknowledgments:** We extend our sincere gratitude to the developers and maintainers of PyTorch 2.12 for providing the deep learning framework essential to this research. We also gratefully acknowledge the availability of the CIC-IDS2017, NSL-KDD, and UNSW-NB15 datasets, which were instrumental in evaluating our proposed models.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## Abbreviations

The following abbreviations are used in this manuscript:

|      |                                                         |
|------|---------------------------------------------------------|
| AI   | Artificial Intelligence                                 |
| BERT | Bidirectional Encoder Representations from Transformers |
| CNN  | Convolutional Neural Network                            |
| DL   | Deep Learning                                           |
| EDR  | Endpoint Detection and Response                         |
| GAN  | Generative Adversarial Network                          |
| GNN  | Graph Neural Network                                    |
| GPT  | Generative Pre-trained Transformer                      |
| IDS  | Intrusion Detection System                              |
| KNN  | K-Nearest Neighbours                                    |
| LLM  | Large Language Model                                    |
| LR   | Logistic Regression                                     |
| ML   | Machine Learning                                        |
| MLP  | Multilayer Perceptron                                   |
| NB   | Naive Bayes                                             |
| RBF  | Radial Basis Function                                   |
| RF   | Random Forest                                           |
| RFE  | Recursive Feature Elimination                           |
| RL   | Reinforcement Learning                                  |
| RNN  | Recurrent Neural Network                                |
| SOC  | Security Operations Centre                              |
| SOAR | Security Orchestration Automation and Response          |
| SVM  | Support Vector Machine                                  |
| XDR  | Extended Detection and Response                         |

## References

1. CrowdStrike, Inc. 2026 Global Threat Report. 2026. Available online: <https://go.crowdstrike.com/2026-global-threat-report.html> (accessed on 4 March 2026).
2. IBM Security. Cost of a Data Breach Report 2025. 2025. Available online: <https://www.ibm.com/reports/data-breach> (accessed on 4 March 2026).
3. Dalal, A. Exploring next-generation cybersecurity tools for advanced threat detection and incident response. *Sci. Technol. Dev.* **2020**, *X*, 535–553. [CrossRef]
4. Maddireddy, B.R.; Maddireddy, B.R. Enhancing Network Security through AI-Powered Automated Incident Response Systems. *Int. J. Adv. Eng. Technol. Innov.* **2023**, *1*, 282–304.
5. Tatineni, S. AI-infused threat detection and incident response in cloud security. *Int. J. Sci. Res.* **2023**, *12*, 998–1004. [CrossRef]
6. Obuse, E.; Etim, E.D.; Essien, I.A.; Cadet, E.; Ajayi, J.O.; Erigha, E.D.; Babatunde, L.A. AI-powered incident response automation in critical infrastructure protection. *Int. J. Adv. Multidiscip. Res. Stud.* **2023**, *3*, 1156–1171. [CrossRef]



7. Dhanushkodi, K.; Thejas, S. AI enabled threat detection: Leveraging artificial intelligence for advanced security and cyber threat mitigation. *IEEE Access* **2024**, *12*, 173127–173136. [\[CrossRef\]](#)
8. Abdulrahman, I.A.; Ogor, U.C.; Ayodele, G.T.; Anadozie, C.; Alebiosu, J. AI-Driven Threat Intelligence and Automated Incident Response: Enhancing Cyber Resilience through Predictive Analytics. *Res. J. Civ. Ind. Mech. Eng.* **2025**, *2*, 16–32. [\[CrossRef\]](#)
9. Tanikonda, A.; Pandey, B.K.; Peddinti, S.R.; Katragadda, S.R. Advanced AI-driven cybersecurity solutions for proactive threat detection and response in complex ecosystems. *J. Sci. Technol.* **2022**, *3*, 196–218. [\[CrossRef\]](#)
10. Nnaka, K.I.; Mbamalu, P.O.; Nwaigbo, J.C.; Ozo-ogweji, P.C.; Njoku, V.I.; Ekechi, C.C. AI-powered threat detection: Opportunities and limitations in modern cyber defense. *World J. Adv. Res. Rev.* **2025**, *27*, 210–223. [\[CrossRef\]](#)
11. Sufyan, A.; Mujeeb-Ur-Rehman, M.; Noreen, B.; Amin, S. Trends, capabilities, and challenges in modern cyber defense: A systematic review of detection and response technologies. *Spectr. Eng. Sci.* **2026**, *4*, 464–503.
12. Ali, B.; Shah, S.I.; Sajid, L.; Talpur, M.R.H.; Javed, M.U.; Warsi, M.U. Design of Intelligent Cyber Defense Frameworks Using Artificial Intelligence for Proactive Threat Detection, Prediction, and Automated Response. *Glob. Res. J. Nat. Sci. Technol.* **2026**, *4*. [\[CrossRef\]](#)
13. Molina-Coronado, B.; Mori, U.; Mendiburu, A.; Miguel-Alonso, J. Survey of network intrusion detection methods from the perspective of the knowledge discovery in databases process. *IEEE Trans. Netw. Serv. Manag.* **2020**, *17*, 2451–2479. [\[CrossRef\]](#)
14. Chirra, D.R. Towards an AI-Driven Automated Cybersecurity Incident Response System. *Int. J. Adv. Eng. Technol. Innov.* **2023**, *1*, 429–451.
15. Yaseen, A. AI-driven threat detection and response: A paradigm shift in cybersecurity. *Int. J. Inf. Cybersecur.* **2023**, *7*, 25–43.
16. Chen, Y.; Cui, M.; Wang, D.; Cao, Y.; Yang, P.; Jiang, B.; Lu, Z.; Liu, B. A survey of large language models for cyber threat detection. *Comput. Secur.* **2024**, *145*, 104016. [\[CrossRef\]](#)
17. Khan, M.I.; Arif, A.; Khan, A.R.A. The most recent advances and uses of AI in cybersecurity. *Bullet J. Multidisiplin Ilmu* **2024**, *3*, 566–578.
18. Okoli, U.I.; Obi, O.C.; Adewusi, A.O.; Abrahams, T.O. Machine learning in cybersecurity: A review of threat detection and defense mechanisms. *World J. Adv. Res. Rev.* **2024**, *21*, 2286–2295. [\[CrossRef\]](#)
19. Alzaabi, F.R.; Mehmood, A. A review of recent advances, challenges, and opportunities in malicious insider threat detection using machine learning methods. *IEEE Access* **2024**, *12*, 30907–30927. [\[CrossRef\]](#)
20. Tsai, C.F.; Hsu, Y.F.; Lin, C.Y.; Lin, W.Y. Intrusion detection by machine learning: A review. *Expert Syst. Appl.* **2009**, *36*, 11994–12000. [\[CrossRef\]](#)
21. Stiawan, D.; Idris, M.Y.B.; Bamhdi, A.M.; Budiarto, R. CICIDS-2017 dataset feature analysis with information gain for anomaly detection. *IEEE Access* **2020**, *8*, 132911–132921. [\[CrossRef\]](#)
22. Moustafa, N.; Slay, J. UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS)*; IEEE: Canberra, Australia 2015; pp. 1–6.
23. Solarmainframe. IDS Intrusion CSVs (CSE-CIC-IDS2018). 2018. Kaggle Dataset. Available online: <https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv> (accessed on 5 February 2026).
24. Hussain, A.A.; Ahmad, M.; Sajjad, F.; Ali, M.; Bajwa, M.T.T.; Elahi, H. AI-Driven Intrusion Detection System for Future 5G Networks. *Spectr. Eng. Sci.* **2026**, *4*, 1303–1319.
25. Quan, N.W.; Goh, H.N.; Lim, A.H.L. Artificial Intelligence-Based Intrusion Detection System Through Ensemble Approaches. In *Proceedings of the AIP Conference Proceedings*; AIP Publishing LLC: Melville, NY, USA, 2025; Volume 3367, p. 020003.
26. Ahmed, U.; Jiangbin, Z.; Almogren, A.; Khan, S.; Sadiq, M.T.; Altameem, A.; Rehman, A.U. Explainable AI-Based Innovative Hybrid Ensemble Model for Intrusion Detection. *J. Cloud Comput.* **2024**, *13*, 150. [\[CrossRef\]](#)
27. Mohammad, M.F.; Elmedany, W.; Sharif, M.S. Hybrid AI-Driven Intrusion Detection Systems. In *Joint International Conference on AI, Big Data and Blockchain*; Springer Nature: Cham, Switzerland, 2025; pp. 178–194.
28. Purushothaman, S.; Suhashini, A.; Thakur, A.; Anandhi, K.; Bhatnagar, S. Optimized IDS Using AI and Feature Ranking Fusion for Enhanced Threat Detection. In *Proceedings of the 2025 IEEE 1st International Conference on Smart Innovations in Systems, Infrastructure, Mechanical, Power, AI and Computing Technologies (SISIMPACT)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1250–1255.
29. Musthafa, M. Adversarial Robustness in AI-Driven Cybersecurity Solutions: Thwarting Evasion Assaults in Real-Time Detection Systems. *Int. J. Adv. Eng. Manag. Sci.* **2025**, *11*, 632791. [\[CrossRef\]](#)
30. Alkhater, N. A Rigorous Comparative Study of Supervised Machine Learning Techniques for Network Anomaly Detection: Empirical Insights from the UNSW-NB15 Dataset. *Computers* **2026**, *15*, 285. [\[CrossRef\]](#)
31. Abdulqadder, I.H.; Zhou, S.; Zou, D.; Aziz, I.T.; Akber, S.M.A. Multi-layered Intrusion Detection and Prevention in the SDN/NFV Enabled Cloud of 5G Networks Using AI-Based Defense Mechanisms. *Comput. Netw.* **2020**, *179*, 107364. [\[CrossRef\]](#)
32. Lansky, J.; Ali, S.; Mohammadi, M.; Majeed, M.K.; Karim, S.H.T.; Rashidi, S.; Hosseinzadeh, M.; Rahmani, A.M. Deep learning-based intrusion detection systems: A systematic review. *IEEE Access* **2021**, *9*, 101574–101599. [\[CrossRef\]](#)

33. Bakhsh, S.A.; Khan, M.A.; Ahmed, F.; Alshehri, M.S.; Ali, H.; Ahmad, J. Enhancing IoT network security through deep learning-powered Intrusion Detection System. *Internet Things* **2023**, *24*, 100936. [CrossRef]
34. Kheddar, H. Transformers and large language models for efficient intrusion detection systems: A comprehensive survey. *Inf. Fusion* **2025**, *124*, 103347. [CrossRef]
35. Dunmore, A.; Jang-Jaccard, J.; Sabrina, F.; Kwak, J. A comprehensive survey of generative adversarial networks (GANs) in cybersecurity intrusion detection. *IEEE Access* **2023**, *11*, 76071–76094. [CrossRef]
36. Bilot, T.; El Madhoun, N.; Al Agha, K.; Zouaoui, A. Graph neural networks for intrusion detection: A survey. *IEEE Access* **2023**, *11*, 49114–49139. [CrossRef]
37. Sewak, M.; Sahay, S.K.; Rathore, H. Deep reinforcement learning in the advanced cybersecurity threat detection and protection. *Inf. Syst. Front.* **2023**, *25*, 589–611. [CrossRef]
38. Sewak, M.; Sahay, S.K.; Rathore, H. Deep reinforcement learning for cybersecurity threat detection and protection: A review. In *Proceedings of the International Conference on Secure Knowledge Management in Artificial Intelligence Era*; Springer: Cham, Switzerland, 2021; pp. 51–72.
39. Arshad, K.; Ali, R.F.; Muneer, A.; Aziz, I.A.; Naseer, S.; Khan, N.S.; Taib, S.M. Deep reinforcement learning for anomaly detection: A systematic review. *IEEE Access* **2022**, *10*, 124017–124035. [CrossRef]
40. Xu, H.; Wang, S.; Li, N.; Wang, K.; Zhao, Y.; Chen, K.; Yu, T.; Liu, Y.; Wang, H. Large language models for cyber security: A systematic literature review. *ACM Trans. Softw. Eng. Methodol.* **2025**. [CrossRef]
41. Gupta, M.; Akiri, C.; Aryal, K.; Parker, E.; Praharaj, L. From chatgpt to threatgpt: Impact of generative AI in cybersecurity and privacy. *IEEE Access* **2023**, *11*, 80218–80245. [CrossRef]
42. SANS Institute. SANS 504-B Incident Response Cycle: Cheat Sheet. 2016. Available online: <https://www.sans.org/media/score/504-incident-response-cycle.pdf> (accessed on 6 March 2026).
43. Nelson, A.; Rekhi, S.; Souppaya, M.; Scarfone, K. *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*; NIST Special Publication SP 800-61 Rev. 3; National Institute of Standards and Technology (NIST): Gaithersburg, MD, USA, 2025. [CrossRef]
44. Falowo, O.I.; Bou Abdo, J. Empirical study on automation, ai trust, and framework readiness in cybersecurity incident response. *Algorithms* **2026**, *19*, 62. [CrossRef]
45. Polinati, A.K. AI and Deep Learning-Powered Threat Intelligence and Automated Response Mechanisms. In *Proceedings of the 2025 3rd International Conference on Sustainable Computing and Data Communication Systems (ICSCDS)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1504–1509.
46. Kinyua, J.; Awuah, L. AI/ML in Security Orchestration, Automation and Response: Future Research Directions. *Intell. Autom. Soft Comput.* **2021**, *28*, 527–545. [CrossRef]
47. Mir, A.W.; Ramachandran, R.K. Implementation of security orchestration, automation and response (SOAR) in smart grid-based SCADA systems. In *Proceedings of the Sixth International Conference on Intelligent Computing and Applications: Proceedings of ICICA 2020*; Springer: Cham, Switzerland, 2021; pp. 157–169.
48. Mohsin, A.; Janicke, H.; Ibrahim, A.; Sarker, I.H.; Camtepe, S. A unified framework for human ai collaboration in security operations centers with trusted autonomy. *arXiv* **2025**, arXiv:2505.23397. [CrossRef]
49. Cichonski, P.; Millar, T.; Grance, T.; Scarfone, K. *Computer Security Incident Handling Guide*; NIST Special Publication SP 800-61 Rev. 2; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2012. [CrossRef]
50. Albarrak, M.; Salonitis, K.; Jagtap, S. Natural language processing (NLP)-based frameworks for cyber threat intelligence and early prediction of cyberattacks in Industry 4.0: A systematic literature review. *Appl. Sci.* **2026**, *16*, 619. [CrossRef]
51. Jumani, A.; Baig, A.; Akhtar, E.D.S.; Shamim, M.S.; Zaheer, H.; Changaiz, A. Automating cyber threat intelligence extraction using natural language processing techniques. *Kashf J. Multidiscip. Res.* **2025**, *2*, 184–201. [CrossRef]
52. Sharma, S.; Arjunan, T. Natural language processing for detecting anomalies and intrusions in unstructured cybersecurity data. *Int. J. Inf. Cybersecur.* **2023**, *7*, 1–24.
53. Klein, T.; Romano, G. Optimizing Cybersecurity Incident Response via Adaptive Reinforcement Learning. *J. Adv. Eng. Technol.* **2025**, *2*. [CrossRef]
54. Cadet, E.; Etim, E.D.; Essien, I.A.; Ajayi, J.O.; Erigha, E.D. The role of reinforcement learning in adaptive cyber defense mechanisms. *Int. J. Multidiscip. Res. Growth Eval.* **2021**, *2*, 544–559. [CrossRef]
55. Alturkistani, H.; Jaafar, A.G.; Chuprat, S. Automating Cyber Threat Intelligence Workflows with LLMs: Processing, Analyzing, and Defending in One Model. In *Proceedings of the 2025 3rd International Conference on Cyber Resilience (ICCR)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1–6.
56. Kumar, V.N.; Kumar, M.S.; Suhasini, R.; Lakshman, M.; Anbalagan, N.; D. R., K. A Graph Neural Network Framework for Real Time Cyber Threat Intelligence and Risk Analysis. In *Proceedings of the 2025 IEEE International Conference on Advanced Computing Technologies (ICACT)*; IEEE: Piscataway, NJ, USA, 2025; pp. 630–636.

57. Hasan, M.M.; Nijhum, A.M. Deep Learning And Graph Neural Networks For Real-Time Cybersecurity Threat Detection. *Rev. Appl. Sci. Technol.* **2024**, *3*, 106–142. [\[CrossRef\]](#)
58. Sarhan, M.; Layeghy, S.; Moustafa, N.; Portmann, M. Cyber threat intelligence sharing scheme based on federated learning for network intrusion detection. *J. Netw. Syst. Manag.* **2023**, *31*, 3. [\[CrossRef\]](#)
59. Saddi, V.R.; Gopal, S.K.; Mohammed, A.S.; Dhanasekaran, S.; Naruka, M.S. Examine the Role of Generative AI in Enhancing Threat Intelligence and Cyber Security Measures. In *Proceedings of the 2024 2nd International Conference on Disruptive Technologies (ICDT)*; IEEE: Piscataway, NJ, USA, 2024; pp. 537–542. [\[CrossRef\]](#)
60. Patel, A.; Pandey, P.; Ragothaman, H.; Molleti, R.; Peddinti, D.R. Generative AI for Automated Security Operations in Cloud Computing. In *Proceedings of the 2025 IEEE 4th International Conference on AI in Cybersecurity (ICAIC)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1–7.
61. Esseghir, A.; Kamoun, F.; Hraiech, O. AKER: An open-source security platform integrating IDS and SIEM functions with encrypted traffic analytic capability. *J. Cyber Secur. Technol.* **2022**, *6*, 27–64. [\[CrossRef\]](#)
62. Alharbi, S.; Khan, A. Ensemble Defense System: A Hybrid IDS Approach for Effective Cyber Threat Detection. In *Proceedings of the 2023 33rd International Telecommunication Networks and Applications Conference*; IEEE: Piscataway, NJ, USA, 2023; pp. 267–270.
63. Muhammad, A.R.; Sukarno, P.; Wardana, A.A. Integrated security information and event management (siem) with intrusion detection system (ids) for live analysis based on machine learning. *Procedia Comput. Sci.* **2023**, *217*, 1406–1415. [\[CrossRef\]](#)
64. Dalwai, A.A.; Jaswal, S.; Verma, R. Securing IoT System Using ML Models. In *Optimizing Edge and Fog Computing Applications with AI and Metaheuristic Algorithms*; Auerbach Publications: Boca Raton, FL, USA, 2025; pp. 84–112.
65. Laboratory, S. StratosphereLinuxIPS (Slips): Machine Learning-Based Intrusion Prevention System. 2025. Available online: <https://github.com/stratosphereips/StratosphereLinuxIPS> (accessed on 10 May 2026).
66. Mareedu, A. Machine Learning Applications in Intrusion Detection: A Comprehensive Review. *Int. J. Multidiscip. Sci. Manag.* **2024**, *1*, 66–78.
67. Nour, B.; Pourzandi, M.; Debbabi, M. A survey on threat hunting in enterprise networks. *IEEE Commun. Surv. Tutor.* **2023**, *25*, 2299–2324. [\[CrossRef\]](#)
68. Dey, A. Datascience in Support of Cybersecurity Operations: Adaptable, Robust and Explainable Anomaly Detection for Security Analysts. Ph.D. Thesis, Ecole Nationale sup rieure Mines-T l com Atlantique, Brest, France, 2022.
69. Pissanidis, D.L.; Demertzis, K. Integrating AI/ML in cybersecurity: An analysis of open XDR technology and its application in intrusion detection and system log management. *Preprints* **2023**. [\[CrossRef\]](#)
70. Heino, T.; Mohammad, T.; Hakkala, A. Real-Time Threat Detection using SIEM for Industrial IoT Protocols. Master’s Thesis, University of Turku, Turku, Finland, 2025.
71. Kalodanis, K.; Papapavlou, C.; Feretzakis, G. Enhancing Security in 5G and Future 6G Networks: Machine Learning Approaches for Adaptive Intrusion Detection and Prevention. *Future Internet* **2025**, *17*, 312. [\[CrossRef\]](#)
72. Tumparthy, N. Efficient Intrusion Detection for Smart Homes: Suricata and Machine Learning for Speed and Efficiency. Master’s Thesis, National College of Ireland, Dublin, Ireland, 2025.
73. Adrovi , H. Enhancing Smart Home Security Through IoT Device Fingerprinting Using Machine Learning. Master’s Thesis, M lardalen University, The School of Innovation, Design and Engineering V ster s, Sweden, 2025.
74. Aramide, O.O. AI-driven automated incident response and remediation in networks. *Int. J. Technol. Manag. Humanit.* **2025**, *11*, 1–9. [\[CrossRef\]](#)
75. Emiro lu, B.G. AI-driven threat detection and response systems: Enhancing cybersecurity in the digital era. In *Challenges and Solutions for Cybersecurity and Adversarial Machine Learning*; IGI Global Scientific Publishing: Hershey, PA, USA, 2025; pp. 227–270.
76. Charla, R.R. AI-Enhanced Automated Incident Response in SIEM with Explainability for SOC Analysts. In *Proceedings of the 2025 20th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1–10.
77. Huang, X.; Zhang, L.; Wang, B.; Li, F.; Zhang, Z. Feature clustering based support vector machine recursive feature elimination for gene selection. *Appl. Intell.* **2018**, *48*, 594–607. [\[CrossRef\]](#)
78. Panigrahi, R.; Borah, S. A detailed analysis of CICIDS2017 dataset for designing Intrusion Detection Systems. *Int. J. Eng. Technol.* **2018**, *7*, 479–482.
79. Dhanabal, L.; Shantharajah, S. A study on NSL-KDD dataset for intrusion detection system based on classification algorithms. *Int. J. Adv. Res. Comput. Commun. Eng.* **2015**, *4*, 446–452.
80. Tavallae, M.; Bagheri, E.; Lu, W.; Ghorbani, A.A. A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*; IEEE: Piscataway, NJ, USA, 2009; pp. 1–6.
81. Liang, L.L.; Wan, Z.H.; Ye, C.C.; Zhang, P.J.Y.; Sun, D.J.; Lu, X.Y. Flow dynamics and noise generation mechanisms in supersonic underexpanded rectangular and planar jets. *AIP Adv.* **2023**, *13*, 065128. [\[CrossRef\]](#)
82. Elsayed, M.S.; Le-Khac, N.A.; Jurcut, A.D. InSDN: A novel SDN intrusion dataset. *IEEE Access* **2020**, *8*, 165263–165284. [\[CrossRef\]](#)

83. Selvam, R.; Velliangiri, S. An improving intrusion detection model based on novel CNN technique using recent CIC-IDS datasets. In *Proceedings of the 2024 International Conference on Distributed Computing and Optimization Techniques (ICDCOT)*; IEEE: Piscataway, NJ, USA, 2024; pp. 1–6.
84. Peng, C.; Wu, X.; Yuan, W.; Zhang, X.; Zhang, Y.; Li, Y. MGRFE: Multilayer recursive feature elimination based on an embedded genetic algorithm for cancer classification. *IEEE/ACM Trans. Comput. Biol. Bioinform.* **2019**, *18*, 621–632. [[CrossRef](#)] [[PubMed](#)]

**Disclaimer/Publisher’s Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.